CAPSTONE VIVA PREPARATION

EduEarth Questions and Answers

Easy to hard viva questions based on the actual EduEarth project repository: frontend, backend, database, authentication, games, APIs, Docker, metrics, and Airflow ingestion.

PROJECT
Gamified environmental education platform

PREPARED FOR
Capstone viva and project defense

QUESTION COUNT
128 questions with answers

ORDER
Easy, medium, then hard

Best Opening Answer

Use this when the examiner asks: "Explain your project briefly."

Short project introduction

EduEarth is a full-stack gamified environmental education platform for students, teachers, and institutions. It provides learning modules, dashboards, environmental articles, quizzes, and games such as Recycle Rush, Eco Strike, and Eco Sprint. The frontend is built with Next.js and React, the backend uses Express, Prisma, PostgreSQL, Supabase authentication, and Socket.io, while Python Airflow handles environmental data ingestion. The aim is to make sustainability education more interactive, practical, and measurable.

Simple architecture explanation

The user interacts with the Next.js frontend. The frontend calls Express REST APIs and sends the Supabase access token for protected routes. Express validates the token, uses Prisma to work with PostgreSQL, and sends structured responses. Socket.io handles real-time game events. Airflow separately fetches climate and environmental news data, enriches it, stores it in PostgreSQL, and sends update events through an outbox table.

Final viva closing line

EduEarth shows how modern web technologies can be used for a social and educational purpose. The project combines learning, gamification, role-based dashboards, real-time interaction, and data ingestion to make environmental education more engaging.

Tip: In viva, answer first in simple words, then mention the exact technology or file if asked deeper.

Easy Questions

Basic project understanding and quick viva warm-up

1 What is EduEarth?

EduEarth is a gamified environmental education platform for students, teachers, and institutions. It combines learning modules, dashboards, environmental articles, quizzes, and games to make sustainability education more interactive.

2 What problem does your project solve?

Many students learn environmental topics only through theory. EduEarth solves this by combining content, activities, gamification, articles, and real-time games so learners can understand environmental topics in a practical and engaging way.

3 Why did you choose this project?

I chose EduEarth because environmental awareness is important and students learn better when education is interactive. The project also allowed me to work on full-stack development, authentication, database design, real-time features, and data ingestion.

4 Who are the main users of EduEarth?

The main users are students, teachers, and institutions. Students learn and play, teachers manage modules and progress, and institutions can organize classes and educational activities.

5 What are the main modules in EduEarth?

The main modules are authentication, student dashboard, teacher dashboard, learning modules, articles, games, classes, lessons, quizzes, institutions, and environmental ingestion.

6 What is the main objective of EduEarth?

The objective is to promote environmental education through a platform that is interactive, role-aware, and gamified. It helps students learn sustainability topics and track their progress.

7 What technology stack did you use?

The client uses Next.js, React, TypeScript, Tailwind CSS, Framer Motion, and Socket.io client. The server uses Node.js, Express, TypeScript, Prisma, PostgreSQL, Supabase Auth, Socket.io, optional Redis, Docker, and Prometheus metrics. The ingestion side uses Python and Airflow.

8 What is the frontend framework?

The frontend is built with Next.js 15 and React 19 using the App Router structure.

9 **What is the backend framework?**

The backend is built with Express 5 on Node.js using TypeScript.

10 **Which database is used?**

PostgreSQL is used as the main database, accessed through Prisma ORM. In local or hosted setup, Supabase Postgres can be used.

11 **What is Prisma?**

Prisma is an ORM that helps the server interact with the PostgreSQL database using type-safe models instead of writing raw SQL for every query.

12 **What is Supabase used for?**

Supabase is used for authentication. It handles sign-up, sign-in, magic links, Google OAuth, and token validation.

13 **What is gamification in your project?**

Gamification means adding game-like elements such as eco points, levels, badges, streaks, leaderboards, challenges, and quiz games to motivate students.

14 **Name the games in EduEarth.**

EduEarth includes Recycle Rush, Eco Strike, and Eco Sprint.

15 **What is Recycle Rush?**

Recycle Rush is a waste sorting game where users sort waste items into correct bins and earn points.

16 **What is Eco Strike?**

Eco Strike is a quiz battle game where correct answers trigger attacks and the player tries to win by answering environmental questions.

17 **What is Eco Sprint?**

Eco Sprint is a real-time eco quiz race powered by Socket.io where players answer questions quickly and compete with another player.

18 **What is the purpose of the student dashboard?**

The student dashboard shows progress, modules, assignments, missions, leaderboard, messages, certificates, eco points, streaks, badges, and learning activity.

19 What is the purpose of the teacher dashboard?

The teacher dashboard helps teachers manage modules, view student progress, create assignments or missions, and monitor class learning.

20 What are environmental articles in EduEarth?

Environmental articles are news or educational stories stored in the database and shown in the frontend. They can include headline, body, source, section, image URL, publish date, and AI summary.

21 What is a REST API?

A REST API exposes backend resources through HTTP endpoints like GET, POST, PUT, PATCH, and DELETE. EduEarth uses REST APIs for auth, articles, classes, lessons, quizzes, and institutions.

22 What is Socket.io used for?

Socket.io is used for real-time communication, especially for game events and outbox-based realtime updates.

23 What is Airflow used for?

Airflow is used to schedule and orchestrate environmental data ingestion from climate APIs and RSS feeds.

24 What is Docker used for?

Docker is used to containerize the client and server so the project can run consistently across machines and deployment environments.

25 What is a leaderboard?

A leaderboard ranks students based on eco points, completed lessons, challenges, activities, or streaks.

26 What are eco points?

Eco points are reward points earned by completing lessons, quizzes, challenges, activities, and game tasks.

27 What is the use of badges?

Badges reward students for achievements such as learning progress, participation, eco activities, and consistency.

28 What is the role of institutions?

Institutions represent schools, colleges, or universities. They group students, teachers, classes, and leaderboards.

29 What is the role of classes?

Classes connect students and teachers under an institution and support class-wise learning and statistics.

30 **How would you explain EduEarth in one minute?**

EduEarth is a full-stack environmental learning platform. It helps students learn sustainability through modules, articles, quizzes, and games. Teachers can manage learning activities, while the backend handles auth, APIs, database records, real-time game events, and ingestion pipelines.

Medium Questions

Architecture, implementation, database, APIs, and workflows

31 Explain the architecture of EduEarth.

EduEarth has four main layers: Next.js frontend, Express REST and Socket.io backend, PostgreSQL database through Prisma, and Python/Airflow ingestion. The frontend calls REST APIs for stored data and uses WebSockets for real-time game and update events.

32 Why did you separate frontend and backend?

Separation improves maintainability. The frontend focuses on UI and user experience, while the backend handles authentication, database operations, APIs, real-time events, and business logic.

33 How does the client call the backend?

The client uses a shared API helper in `client/lib/api.ts`. It builds requests using the server base URL and adds the Supabase bearer token when a session exists.

34 How does role-based routing work?

The AuthProvider reads the Supabase session and loads the server user. Based on the role, students go to the student dashboard and teachers go to the teacher dashboard.

35 How does authentication work in your project?

The user signs in through Supabase. The frontend receives a session token. Protected backend routes use the token in the Authorization header. The backend validates it using Supabase and syncs the user into the Prisma database.

36 What does the validToken middleware do?

It extracts the access token, validates it with Supabase, syncs or creates the Prisma user, attaches the user to the request, and allows protected routes to continue.

37 Why do you sync Supabase users into Prisma?

Supabase manages authentication, but the application needs its own user data, roles, student profile, teacher profile, eco points, and relations. Prisma stores that application-specific data.

38 What is the purpose of the auth cache?

The backend caches validated tokens for a short time, around 60 seconds, so repeated requests do not need to validate the same token with Supabase every time.

39 What are the important User model fields?

Important fields include id, email, phone, name, avatar, dateOfBirth, isAdmin, role, student relation, teacher relation, institutions, notifications, and refresh tokens.

40 How are students and teachers modeled?

Student and Teacher are separate models linked one-to-one with User. Student stores eco points, level, streak, lesson completions, quiz attempts, badges, and activities. Teacher stores institution and class relations.

41 How is many-to-many enrollment handled?

StudentClass connects students to classes and TeacherClass connects teachers to classes. These join tables allow many students and teachers to be linked to many classes.

42 Why are unique constraints used?

Unique constraints prevent duplicate records, such as duplicate emails, duplicate article URLs, duplicate class enrollment for the same student, and duplicate quiz answers in one attempt.

43 How are articles fetched?

The articles page calls `/articles`, receives paginated data, sorts it by extracted date, and displays cards with image, headline, summary/body preview, source, and publish dates.

44 How does article pagination work?

The backend reads page and limit query parameters, calculates skip, uses Prisma `findMany` with skip and take, and returns pagination metadata such as total pages, hasNext, and hasPrev.

45 How does article filtering work?

The backend supports filters like section, source, search text, sortBy, and sortOrder. It builds a Prisma where clause and orderBy object based on query parameters.

46 Why did you add caching for articles?

Article lists and statistics are read often and do not need to be recalculated for every request. Redis caching reduces database load and improves response speed.

47 What happens if Redis is not configured?

The cache service returns null and the application continues using the database directly. This makes Redis optional instead of a hard dependency.

48 What is Prometheus metrics support?

The backend exposes metrics such as HTTP request duration, database query duration, and cache hit/miss counters using `prom-client`.

49 What is the purpose of `/health` and `/metrics`?

`/health` confirms that the API server is running. `/metrics` exposes runtime metrics for monitoring tools like Prometheus.

50 How does the real-time game matching work?

When a player joins the queue, the server either waits for another player or pairs them with an existing waiting player. Then it creates a game session with selected questions.

51 Where are game sessions stored?

Current game sessions are stored in server memory using Maps and arrays. This is simple for one server instance but needs shared storage for horizontal scaling.

52 How is the winner decided in Eco Sprint?

Players answer selected quiz questions. Correct answers give score bonuses based on speed. At the end, the player with the higher score is declared the winner.

53 How does the server handle a player disconnecting?

The disconnected player is removed from the waiting queue. If the player was in an active game, the opponent is notified and the game session is deleted.

54 What is an outbox pattern?

The outbox pattern stores events in a database table first and then publishes them later. This improves reliability because events are not lost if the WebSocket publisher fails temporarily.

55 How does the outbox publisher work?

The service polls unsent rows from `websocket_outbox`, emits the event through Socket.io, and marks successfully emitted rows as sent.

56 Why is `FOR UPDATE SKIP LOCKED` useful?

It lets one publisher safely claim unsent outbox rows without blocking other workers. It helps avoid two workers publishing the same event.

57 What is the environmental ingestion pipeline?

It is an Airflow DAG that fetches climate data and environmental RSS feeds, enriches article records, upserts them into PostgreSQL, and writes real-time events into the outbox.

58 Why did you decouple ingestion from the Express API?

External API calls can be slow, rate-limited, or fail. Airflow is better for scheduled scraping, retries, backoff, and tracking. Express can then serve stored data quickly.

59 Which external providers are configured?

The project includes Open-Meteo for climate data and RSS sources such as Guardian Environment, UNEP RSS, and NASA Earth RSS.

60 What does curriculum enrichment do?

It converts raw article text into classroom-friendly metadata such as curriculum topic, grade band, difficulty level, learning objective, curriculum tags, and relevance score.

61 How does the ingestion pipeline prevent duplicates?

It creates provider record hashes and uses unique constraints with upsert logic, so the same source record is updated instead of inserted repeatedly.

62 What tables are used in ingestion V2?

The ingestion contract includes `ingestion_runs`, `climate_observations`, `news_articles_ingested`, and `websocket_outbox`.

63 What is the use of ingestion_runs?

`ingestion_runs` stores run metadata such as DAG run id, source, status, fetched count, inserted count, updated count, error count, start time, finish time, and notes.

64 What are lesson modules?

Lesson modules group lessons by category and difficulty, such as climate change, renewable energy, waste management, water conservation, and biodiversity.

65 How is lesson completion stored?

LessonCompletion stores which student completed which lesson, when it was completed, and optional time spent. It has a unique constraint to avoid duplicate completion records.

66 How are quizzes structured?

Quiz contains metadata such as title, passing score, time limit, and eco points. QuizQuestion stores questions, options, correct answer, explanation, points, and order. QuizAttempt stores a student's score and pass/fail result.

67 How are quiz answers stored?

QuizAnswer links an attempt to a question and stores the submitted answer, correctness, and points earned.

68 What is the institution API used for?

It manages institutions and allows fetching institution details, statistics, students, teachers, and classes.

69 What is the class API used for?

It manages classes and supports adding/removing students and teachers, getting class members, and getting class statistics.

70 How are environment variables used?

Environment variables store configuration such as database URL, Supabase URL, Supabase key, client URL, server URL, optional Redis URL, and email/S3 details.

71 Why should secrets not be hard-coded?

Hard-coded secrets are unsafe because they can leak through code repositories. Environment variables keep sensitive configuration outside source code.

72 What is CORS and why is it configured?

CORS controls which frontend origins can call the backend from a browser. EduEarth configures allowed origins such as localhost client URLs and the deployed client URL.

73 What is the use of Docker Compose?

Docker Compose can run multiple services such as the client, server, and optional local database together with one command.

74 How would you run the project locally?

Install dependencies in client and server, set environment variables, run Prisma generate and migrate, start the server on port 6969, and start the Next.js client on port 3000.

75 What is the purpose of TypeScript?

TypeScript adds static typing to JavaScript. It helps catch errors earlier, improves editor support, and makes API and model usage clearer.

76 Why use Next.js App Router?

The App Router provides file-based routing, layouts, client/server component support, and a modern structure for building React applications.

77 Why use Tailwind CSS?

Tailwind CSS helps build responsive interfaces quickly using utility classes and keeps styling consistent across components.

78 Why use Framer Motion?

Framer Motion adds smooth animations and transitions, improving the feel of the game hub, homepage, and interactive UI.

79 **What testing or quality checks are included?**

The server has TypeScript no-emit linting and performance smoke/load scripts. The client has ESLint, build scripts, and Lighthouse CI configuration.

Hard Questions

Deep viva, edge cases, trade-offs, scaling, and improvements

80 What is the biggest technical challenge in EduEarth?

The biggest challenge is integrating multiple systems cleanly: Supabase authentication, Prisma database models, REST APIs, real-time Socket.io games, and Airflow ingestion. The solution is layered architecture with clear responsibility for each component.

81 Why not fetch external environmental data directly from the frontend?

Frontend fetching would expose provider logic, reduce control over caching and validation, and make data inconsistent. Server-side or Airflow ingestion allows validation, persistence, retries, enrichment, and centralized API access.

82 Why move provider fetching from Express to Airflow?

Express should serve user requests quickly. Airflow is designed for scheduled jobs, retries, backoff, monitoring, and batch processing. Moving ingestion to Airflow makes the system more reliable and scalable.

83 How would you scale the real-time game server?

I would use a Socket.io Redis adapter, shared session storage, and a load balancer with sticky sessions or proper pub/sub. Game state should move from memory to Redis or another shared store.

84 What is a limitation of storing active games in memory?

If the server restarts, active games are lost. It also does not work well with multiple backend instances because each instance has its own memory.

85 How would you prevent race conditions in quiz games?

I would validate answer timing on the server, ignore duplicate answers, use game-state flags such as correctFound and timedOut, and for distributed systems use atomic operations in Redis or database transactions.

86 How would you prevent cheating in real-time games?

The server should own the correct answers, scoring, timers, and state transitions. The client should only send selected answers, and the server should validate question id, session id, player id, and timing.

87 How do you handle duplicate users between Supabase and Prisma?

The backend uses the Supabase email as a unique identifier. If a user exists, it updates missing fields; otherwise, it creates a new Prisma user.

88 What are the security risks in your project?

Risks include leaked environment variables, weak role enforcement, insecure CORS, token misuse, unvalidated inputs, and admin-only operations being exposed. These are controlled using environment variables, middleware, role checks, validation, and protected routes.

89 How is admin-only article creation protected?

Create, update, and delete article routes require validToken and check `req.user?.isAdmin`. If the user is not admin, the server returns forbidden.

90 What improvement would you make to role-based authorization?

I would add reusable authorization middleware such as requireRole and requireAdmin, then apply it consistently to protected routes instead of checking roles separately in each controller.

91 How would you improve input validation?

I would use Zod schemas for request bodies and query parameters in each route, returning structured validation errors before database operations.

92 How would you improve article pagination for very large data?

Offset pagination with skip can become slow on large datasets. I would add cursor-based pagination using publishDate and id, plus indexes for common filters.

93 Which indexes are important for articles?

Important indexes include publishDate for sorting, section for filtering, url for uniqueness and lookup, and possibly source and search-related indexes for frequent article queries.

94 Why is cache invalidation important?

If cached data remains after create, update, or delete operations, users may see stale data. EduEarth invalidates article list cache after article mutations.

95 What happens if the cache server fails?

The cache service catches errors and falls back to database reads. The application remains functional, although it may be slower.

96 How would you handle Supabase outage?

Protected login and token validation may fail, so the app should show clear errors, retry safely, and keep public content available where possible. Server-side cache should be short-lived and not bypass security.

97 Why use Prisma instead of raw SQL everywhere?

Prisma gives type safety, model relations, migrations, and simpler CRUD operations. Raw SQL is still useful for special cases such as the outbox query with row locking.

98 Why does the outbox publisher use a transaction?

A transaction ensures that selecting unsent events and marking emitted events as sent happens consistently. It reduces duplicate publishing and keeps event status reliable.

99 What if Socket.io emit fails after selecting outbox rows?

Rows are only marked sent if emit succeeds. Failed rows remain unsent and are retried in a later polling cycle.

100 How would you monitor ingestion lag?

I would track last successful ingestion time, fetched count, error count, provider response time, outbox unsent count, and time between record creation and event sent_at.

101 How does Airflow handle failure?

The DAG has retries and retry delays. Failed tasks can retry independently, and successful source tasks do not need to be repeated unnecessarily if the DAG is designed carefully.

102 What is idempotency and why is it important?

Idempotency means running the same ingestion job multiple times should not create duplicate data. It is important because scheduled jobs may retry after failures.

103 How is idempotency achieved in ingestion?

Provider record ids are generated using hashes, and database unique constraints with upsert logic ensure existing records are updated instead of duplicated.

104 How would you improve data quality in ingestion?

I would validate required fields, normalize dates, filter empty URLs, track source freshness, add schema checks, and alert on unusual record counts or repeated provider failures.

105 How would you connect ingested news to the current Article table?

I would either migrate the frontend to read from `news_articles_ingested` or add a sync job that maps ingested records into the current Article model with headline, body, URL, source, section, image, and summary fields.

106 Why is the API layer read-only for external providers in V2?

It keeps request handling fast and predictable. External provider failures do not directly affect user page loads because the API serves already-ingested historical data.

107 How would you deploy EduEarth?

I would deploy the Next.js client on Vercel or a container platform, deploy the Express server with Docker, use hosted PostgreSQL/Supabase, configure environment variables, and deploy Airflow separately for scheduled ingestion.

108 **How would you secure deployment?**

Use HTTPS, secure environment variables, strict CORS, least-privilege database credentials, protected admin routes, dependency updates, logging, and monitoring.

109 **What would you include in production monitoring?**

I would monitor API latency, error rate, CPU/memory, database query time, cache hit rate, active Socket.io connections, outbox backlog, ingestion success, and frontend performance.

110 **How would you test authentication?**

I would test sign-up, sign-in, magic link, Google OAuth, callback handling, token validation, role assignment, unauthorized requests, and role-based redirects.

111 **How would you test the backend APIs?**

I would write integration tests for each route, including success cases, validation errors, unauthorized access, forbidden admin operations, pagination, and database relation behavior.

112 **How would you test Socket.io games?**

I would use automated clients to join the queue, match players, answer questions, test timeouts, duplicate answers, scoring, game end, and disconnect behavior.

113 **How would you improve teacher dashboard functionality?**

I would connect upload module, assignment, mission, and progress views to real backend APIs, add analytics, grading, feedback, and class-wise reports.

114 **How would you improve student learning experience?**

I would add adaptive recommendations, richer feedback after quizzes, progress roadmaps, certificates, discussion groups, and offline-friendly lesson content.

115 **How would you improve accessibility?**

I would ensure keyboard navigation, semantic HTML, proper labels, color contrast, focus states, alt text for images, and screen-reader friendly status messages.

116 **How would you improve performance on the frontend?**

Optimize images, lazy-load heavy game assets, split code by routes, reduce unnecessary animations, cache API calls, and run Lighthouse audits regularly.

117 **How would you improve database performance?**

Add indexes for common queries, avoid over-fetching, use pagination, analyze slow queries, cache hot reads, and use cursor pagination for large tables.

118 How would you handle schema changes safely?

Use Prisma migrations, test migrations on staging, backfill data carefully, keep backward-compatible API changes, and document changed fields.

119 Why is EduEarth better than a normal LMS for this topic?

A normal LMS mainly serves lessons and assignments. EduEarth focuses on environmental learning with gamification, real-time games, eco points, missions, articles, and sustainability-specific content.

120 What are the current limitations of EduEarth?

Some dashboard elements are UI-focused, real-time game state is in memory, ingestion V2 needs full production integration with article reads, and more automated tests are needed.

121 What future features would you add?

I would add AI-based lesson recommendations, teacher analytics, certificate generation, mobile app support, multilingual content, offline lessons, stronger reporting, and improved multiplayer scaling.

122 How would you explain your database design to examiners?

The database is centered on users and education. Users can become students or teachers, institutions contain classes, classes link students and teachers, lessons and quizzes track learning, and gamification is represented through points, badges, achievements, activities, challenges, and leaderboards.

123 How would you explain the complete data flow?

The user interacts with the Next.js UI. The UI calls Express APIs with a Supabase token. Express validates the token, uses Prisma to read/write PostgreSQL, returns structured JSON, and uses Socket.io for real-time events. Airflow separately ingests environmental data into the database and outbox.

124 What is your personal contribution in this capstone?

My contribution is designing and implementing a full-stack environmental education system, including frontend pages, backend APIs, database schema, authentication flow, games, articles, and ingestion architecture.

125 What did you learn from this project?

I learned full-stack architecture, role-based authentication, database modeling, REST API development, real-time Socket.io logic, Airflow-based ingestion, caching, metrics, Docker workflow, and how to connect technical features with an educational goal.

126 If the examiner asks for the most important feature, what will you say?

The most important feature is the combination of gamified learning and real-time environmental education. EduEarth is not only a content platform; it makes sustainability learning active through games, progress tracking, and live data-ready architecture.

127 If you had only two minutes to demonstrate, what would you show?

I would show login/role flow, student dashboard progress, games page with one game, environmental articles, and then briefly explain the backend APIs, Prisma schema, and Airflow ingestion diagram.

128 What is the final conclusion of EduEarth?

EduEarth shows how technology can make environmental education more engaging and practical. It combines learning, gamification, real-time interaction, and data-driven content in one platform for students, teachers, and institutions.